

In Spring Boot, Dependency Injection (DI) is a design pattern where objects receive their dependencies from an external container rather than creating them internally. This promotes loose coupling, testability, and cleaner code. Spring's IoC (Inversion of Control) container manages beans and their lifecycles. There are three primary ways to inject dependencies:

1. Field Injection (via `@Autowired` on fields)

Dependencies are injected directly into the field using reflection, even if private. This is the simplest and most concise approach but is discouraged in production code because it makes unit testing harder (you can't easily mock dependencies without reflection) and hides dependencies from the constructor.

java

```
@Component
public class UserService {
    @Autowired
    private UserRepository userRepository; // Field injection

    public User getUser(Long id) {
        return userRepository.findById(id);
    }
}
```

2. Setter Injection (via `@Autowired` on setters)

Dependencies are injected through a setter method. This allows optional dependencies and supports re-injection or changing dependencies at runtime. It's less common but useful when a dependency is not mandatory or when you need to perform custom logic after injection.

java

```
@Component
public class EmailService {
    private EmailSender emailSender;

    @Autowired
    public void setEmailSender(EmailSender emailSender) {
        this.emailSender = emailSender;
    }
}
```

```
        System.out.println("EmailSender injected successfully!");
    }

    public void sendEmail(String to, String body) {
        emailSender.send(to, body);
    }
}
```

3. Constructor Injection (via `@Autowired` on constructor – Recommended)

Dependencies are provided through the class constructor. This is the preferred approach because it:

- Makes dependencies immutable (final fields),
- Ensures the bean is never created with missing dependencies (fail-fast),
- Simplifies unit testing (just pass mocks via the constructor),
- Works seamlessly with Spring 4.3+, where `@Autowired` is optional if there's a single constructor.

java

```
@Component
public class OrderService {
    private final PaymentGateway paymentGateway;
    private final NotificationService notificationService;

    // @Autowired is optional here (Spring 4.3+ with single constructor)
    public OrderService(PaymentGateway paymentGateway,
        NotificationService notificationService) {
        this.paymentGateway = paymentGateway;
        this.notificationService = notificationService;
    }

    public void placeOrder(Order order) {
        paymentGateway.process(order);
        notificationService.notifyUser(order.getUserId());
    }
}
```

What exactly is `@Autowired` ?

`@Autowired` is a Spring annotation that tells the IoC container to automatically wire (inject) a matching bean into the annotated field, setter, or constructor. The container resolves dependencies by type (by default). If multiple beans of the same type exist, you can use `@Qualifier("beanName")` alongside `@Autowired` to disambiguate. It can also be used on arrays, collections, and maps to inject all beans of a certain type.

Example with `@Qualifier`:

java

```
@Autowired
@Qualifier("standardDiscount")
private DiscountStrategy discountStrategy;
```

Summary Table:

Injection Type	Pros	Cons	When to Use
Field	Concise, easy to write	Hard to test, hides dependencies	Quick prototypes (avoid in production)
Setter	Optional deps, reconfigurable	Mutable, more verbose	Optional/mandatory dependencies
Constructor	Immutable, testable, fail-fast	Slightly verbose (but worth it)	Always preferred (production)

Best Practice: Always favor constructor injection for mandatory dependencies and use `@Autowired` for flexibility when you have multiple constructors or need `@Qualifier` disambiguation. This aligns with SOLID principles and makes your code robust and maintainable.